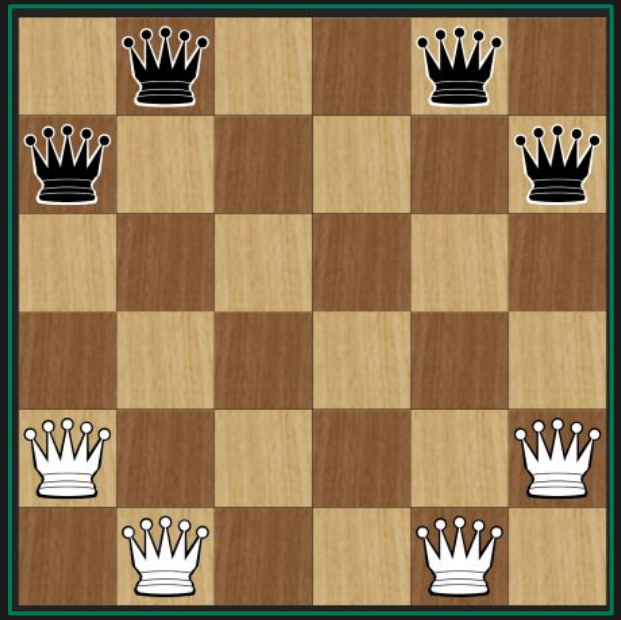
PROJECT PRESENTATION

GAME OF THE AMAZONS

Available on:
https://github.com/ehadziabdic/GameOfTheAmazons

About the Game of Amazons

- Invented in 1988 by Walter Zamkauskas of Argentina
- Strategy game for two players, combining the movement of chess with the added space-restriction of Go.
- Each player controls four Amazons (Queens).
- Battle for space where players try to isolate their opponent by blocking of sections of board with arrows.
- The Objective - be the last player capable of making a move.
- This makes the Game of Amazons a fight for survival.



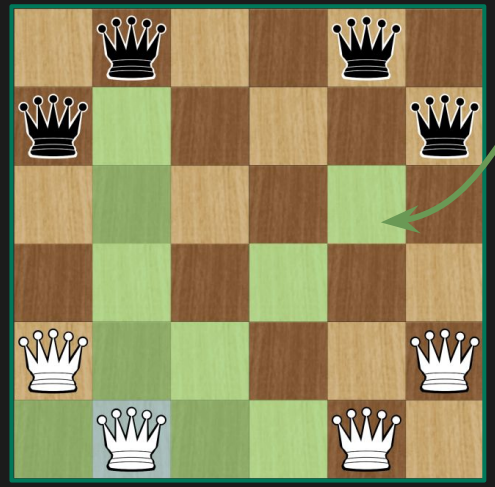
01

GAME RULES

Dev: Muhammed Pašić

Game Rules - Player's Turn

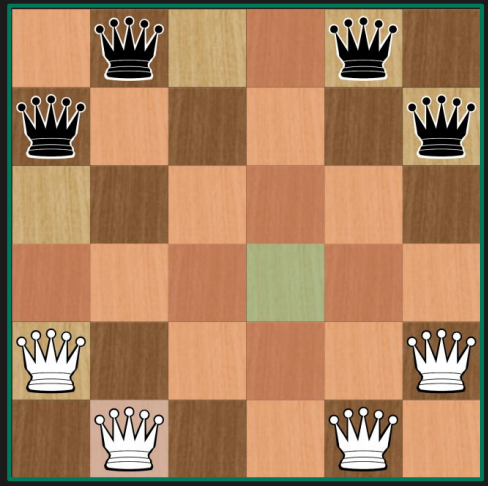
- White goes first.
- Shift one Amazon any distance (vertically, horizontally or diagonally) like a chess queen.
- From this new position, fire your arrow to any empty square using the same queen like movement.



Legal moves for Amazon

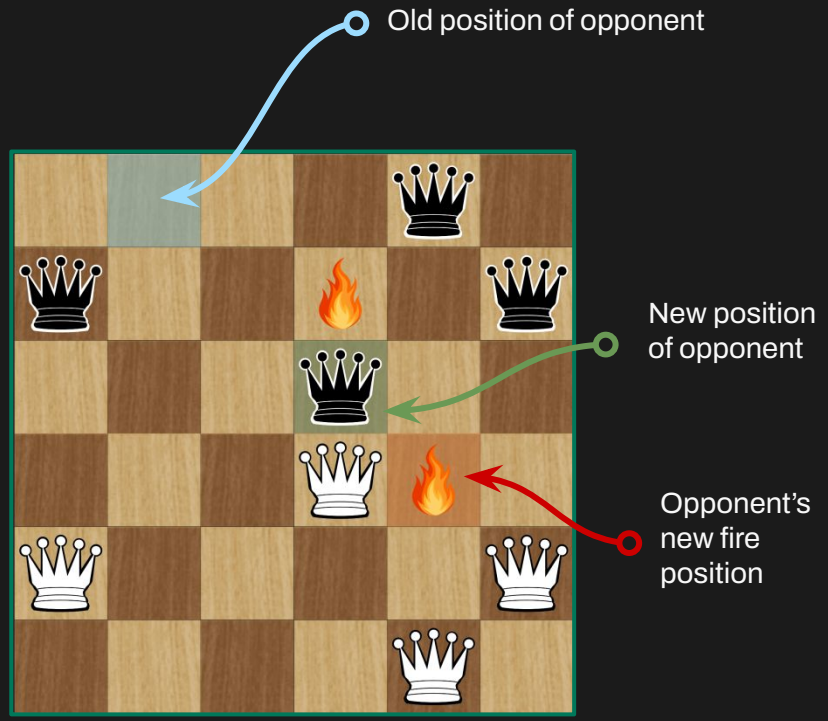
Legal moves for the arrow

Current position



Game Rules - Player's Turn

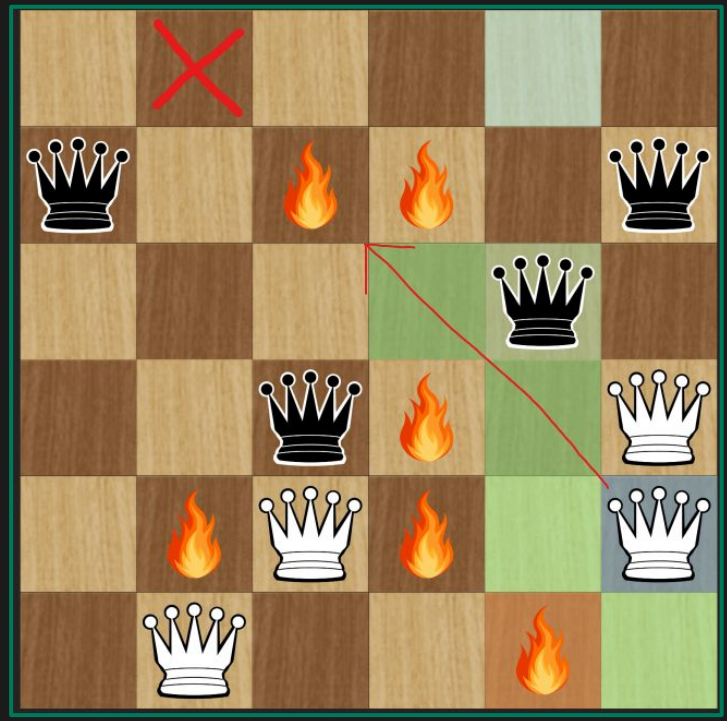
- White goes first.
- Shift one Amazon any distance (vertically, horizontally or diagonally) like a chess queen.
- From this new position, fire your arrow to any empty square using the same queen like movement.



Game Rules - Restrictions

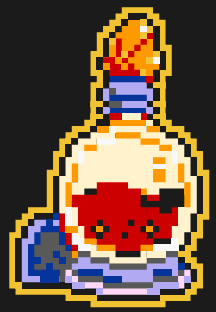
- Once an arrow lands on a square, that square is off limits for the remainder of the game.

- Neither Amazons nor arrows can pass through or jump over blocked or occupied squares.



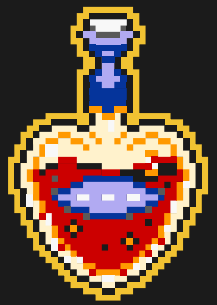
Game Rules - Victory

- The game goes on until a player is walled in and has no legal moves left.



02

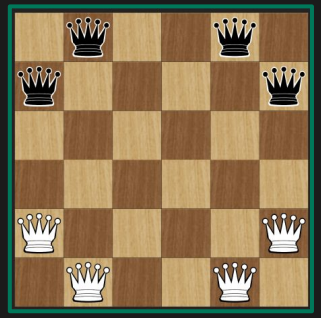
GAME INTERFACE



Dev: Emin Hadžiabdić

Game interface – Board Grid

- ✓ Nested loop iterates every board position
- ✓ **Checkerboard pattern:** `(row + col) % 2`
- ✓ **Image themes:** Draw pre-loaded textures
- ✓ **Color themes:** Use `Shape::drawRect()`
- ✓ **Efficient:** Single draw call per tile

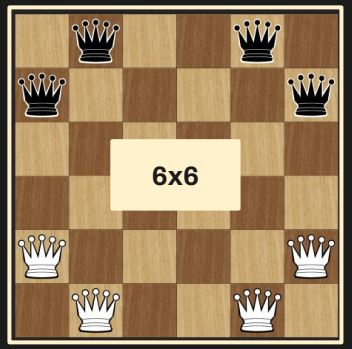


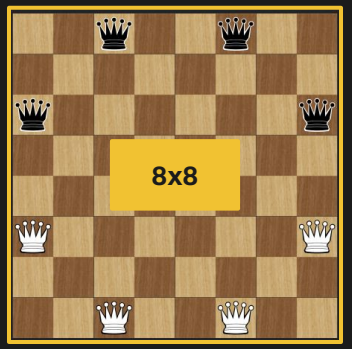
```cpp
void drawBoardGrid() const {
    const int gridSize = _state->boardSize();
    const bool hasImageTheme = (_boardStyle == BoardStyle::Wooden ||
                                _boardStyle == BoardStyle::Ice ||
                                _boardStyle == BoardStyle::Stone ||
                                _boardStyle == BoardStyle::Diamond ||
                                _boardStyle == BoardStyle::Tournament);

    for (int row = 0; row < gridSize; ++row) {
        for (int col = 0; col < gridSize; ++col) {
            Position pos{row, col};
            auto cell = cellRect(pos);
            bool isLightSquare = (row + col) % 2 == 0;

            if (hasImageTheme) {
                if (isLightSquare) {
                    _lightTileImage.draw(cell);
                } else {
                    _darkTileImage.draw(cell);
                }
            } else {
                // Color-based rendering
                td::ColorID color = getColorForSquare(isLightSquare);
                gui::Shape::drawRect(cell, color);
            }
        }
    }
}
```

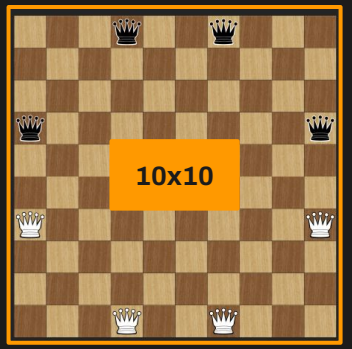
Game interface - Board Sizes

We implemented **three distinct board sizes** to cater to different player skill levels and time constraints.

- ➔ **6x6 (Small):** Designed for fast-paced "Blitz" games
- ➔ **8x8 (Medium):** A balanced, standard competitive size with moderate complexity.
- ➔ **10x10 (Large):** Designed for "Professional" games, offering maximum strategic depth.







Game interface - Board Style

Available Themes:

- → Wooden
- → B&W
- → Tournament
- → Ice
- → Diamond
- → Stone
- → Bubblegum
- → Custom

} Provide **professional** and **traditional** look

} Provide more **modern** feel

→ Exclusive theme just for **female** players

→ “We didn't just give users colors; we gave them a **tool to create their own**.”

Inspired by Chess.com

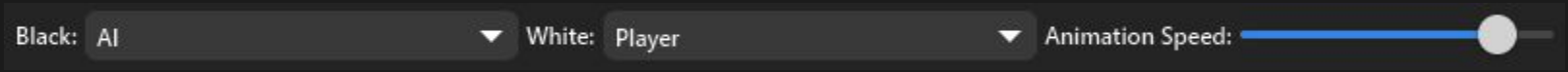
Game interface - Game Modes

Matchmaking Modes:

- ➔ Player vs Player → Local multiplayer for social play.
- ➔ Player vs AI → Test skills against our Minimax engine.
- ➔ AI vs AI → "Spectator Mode" to observe the AI's decision-making

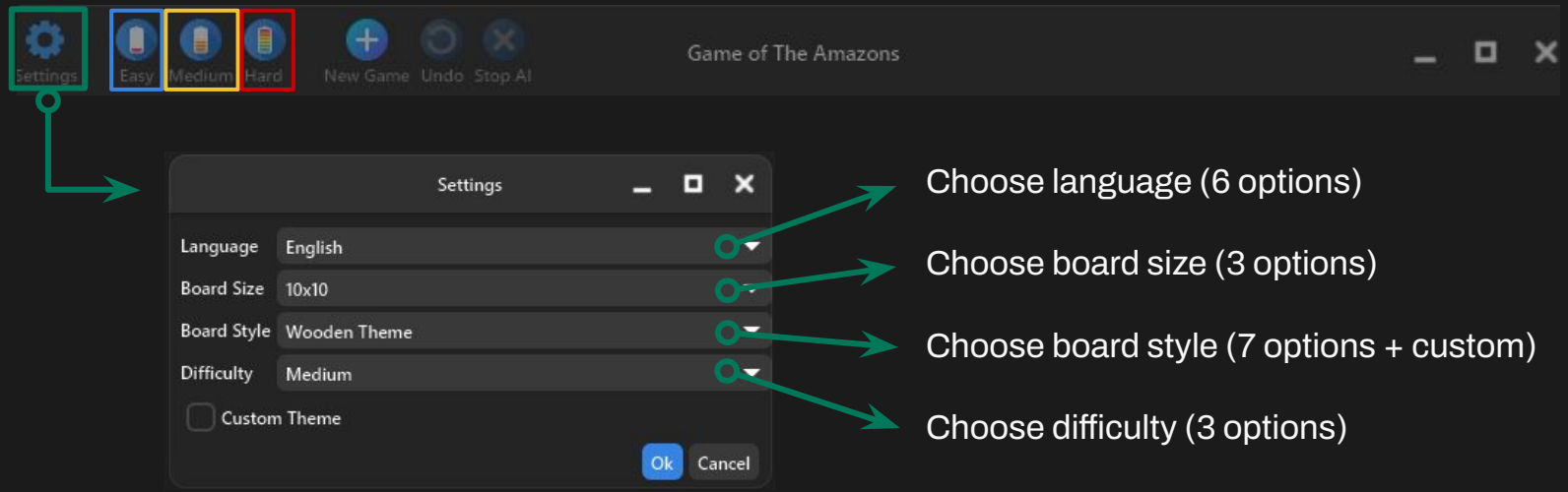
Animation Speed Slider:

A major UX addition that allows users to adjust the **pace for AI vs AI** matches from slow, cinematic movement to near-instantaneous play.



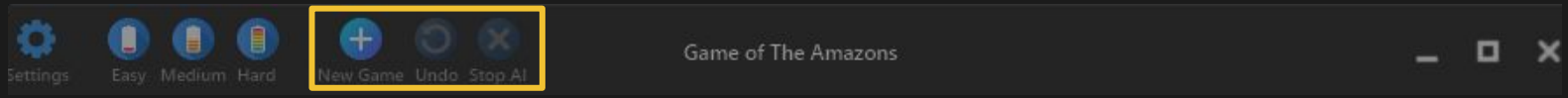
Game interface - Game Controls

- ➔ **Easy:** The AI configured to be Easy Beatable, for beginners.
- ➔ **Medium:** The AI configured to be Hardly Beatable, for intermediate players.
- ➔ **Hard:** The AI configured to be Almost Unbeatable, for an expert-level opponent.

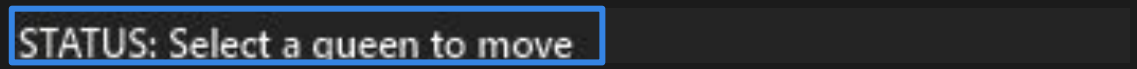


Game interface - Game Controls

- **New Game Button:** Resets the board state
- **Undo Button:** Allows players to **roll back queen** movement if they **don't want to make arrow placement**.
- **Stop AI Button:** Provides a **"safe exit"** by interrupting the **Minimax** search threads.

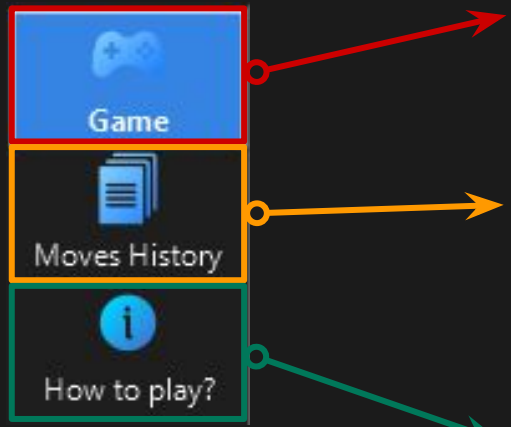


- **Status bar:** Acts as a real-time guide for the player



Game interface - Navigator

We implemented a three-page system to keep the interface clean and organized while providing all necessary information to the player.



Game Page

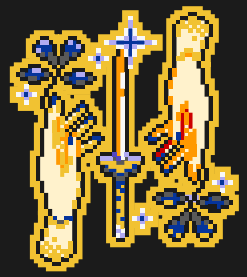
➔ The primary interactive layer where the board is rendered, animations occur (explosions, slides), and the actual gameplay takes place.

Move History Page

➔ A log that records every coordinate-to-coordinate move and arrow shot. This allows players to review their strategy or see exactly where the AI outmaneuvered them.

Rules & Info Page

➔ An in-game manual that ensures that even players unfamiliar with the Game of the Amazons can start playing immediately without external help.



03

GAME ENGINE

Dev: Armin Memišević

Game Engine – Core Algorithm

➔ Minimax Algorithm

- What: The core decision-making logic that predicts future game states.
- How: It recursively simulates moves. It assumes the AI wants to maximize the score and the opponent wants to minimize it.

➔ Alpha-Beta Pruning

- What: An optimization to make the search faster.
- How: If the AI finds a move that is clearly worse than a previous option, it immediately stops searching that branch (the if (alpha >= beta) break; line).

```cpp
inline int minimax(GameState& state, int depth, int alpha, int beta, Player maximizingPlayer,
    Player perspective, std::size_t moveCap, Difficulty difficulty, const std::atomic_bool* cancel = nullptr) {

    if (cancel && cancel->load()) throw SearchCanceled();

    bool terminal = depth == 0 || detail::isTerminal(state);
    if (terminal) {
        GameState evalState = state;
        evaluateWinState(evalState);
        return detail::evaluateTerminal(evalState, perspective);
    }

    Player current = state.currentPlayer();
    bool isMaximizing = current == maximizingPlayer;
    auto moves = generateMovesForPlayer(state, current, moveCap);

    if (moves.empty()) {
        GameState evalState = state;
        evaluateWinState(evalState);
        return detail::evaluateTerminal(evalState, perspective);
    }

    if (isMaximizing) {
        int value = std::numeric_limits<int>::min();
        for (const auto& move : moves) {
            if (cancel && cancel->load()) throw SearchCanceled();
            GameState next = state.clone();
            applyMove(next, move);
            int child = minimax(next, depth - 1, alpha, beta, maximizingPlayer, perspective, moveCap, difficulty, cancel);
            value = std::max(value, child);
            alpha = std::max(alpha, value);
            if (alpha >= beta) break;
        }
        return value;
    }

    int value = std::numeric_limits<int>::max();
    for (const auto& move : moves) {
        if (cancel && cancel->load()) throw SearchCanceled();
        GameState next = state.clone();
        applyMove(next, move);
        int child = minimax(next, depth - 1, alpha, beta, maximizingPlayer, perspective, moveCap, difficulty, cancel);
        value = std::min(value, child);
        beta = std::min(beta, value);
        if (alpha >= beta) break;
    }
    return value;
}
```

Game Engine – Search Constraints & Difficulty Scaling.

➔ Search Depth

- What: Determines how many moves into the future the AI looks.
- How: * Easy: 2 moves ahead. Fast, but misses long-term traps.
- Medium/Hard: 3 moves ahead. This is the "sweet spot" for high-level play without long wait times.

➔ Move Capping

- What: Limits the "Branching Factor" (the number of moves explored per turn).
- How: * In Amazons, one turn can have hundreds of possibilities. We cap this at 100–150 moves depending on difficulty.
    - Why: This prevents the AI from "over-thinking" irrelevant moves, keeping the performance cost low and the game responsive even on weaker hardware.

```
inline int depthForDifficulty(Difficulty difficulty) {
    switch (difficulty) {
    case Difficulty::Easy: return 2;
    case Difficulty::Medium: return 3;
    case Difficulty::Hard:
    default: return 3;
    }
}

inline std::size_t moveCapForDifficulty(Difficulty difficulty) {
    switch (difficulty) {
    case Difficulty::Easy: return 100;
    case Difficulty::Medium: return 120;
    case Difficulty::Hard:
    default: return 150;
    }
}
```

Game Engine - Heuristic Evaluation

Territory Score (Hard Mode)

- What: Calculates which player "owns" more of the board.
- How: Uses a Flood Fill algorithm (BFS) to count how many tiles our Queens can reach before the opponent.

```cpp
inline int territoryScore(const GameState& state, Player player) {
        Player opponent = getOpponent(player);
        std::size_t playerReachable = 0;
        std::size_t opponentReachable = 0;

        for (const auto& pos : scanForQueens(state, player)) {
            playerReachable += floodFillReachableTiles(state, pos);
        }

        for (const auto& pos : scanForQueens(state, opponent)) {
            opponentReachable += floodFillReachableTiles(state, pos);
        }

        return static_cast<int>(playerReachable) - static_cast<int>(opponentReachable);
    }
```

Spatial Influence (Medium & Hard)

- What: Encourages the AI to control the center of the board.
- How: Calculates distance from the center. Center squares give higher scores; corners give lower scores.

```cpp
inline int spatialInfluenceScore(const GameState& state, Player player) {
        const auto& board = state.board();
        int dim = board.dimension();

        if (dim == 0) return 0;

        double center = (dim - 1) / 2.0;

        auto positionalValue = [&](const Position& pos) {
            // Distance from center
            double dist = std::abs(pos.row - center) + std::abs(pos.col - center);
            double maxDist = 2.0 * (dim - 1);
            // Normalized: 1.0 = Center, 0.0 = Corner
            double normalized = 1.0 - (dist / maxDist);

            // Factor in mobility slightly to ensure the spot isn't a trap
            auto mobility = gatherReachableTiles(board, pos).size();
            return static_cast<double>(mobility) * 0.25 + normalized * 10.0;
        };
```

Game Engine - Performance & Difficulty Handling

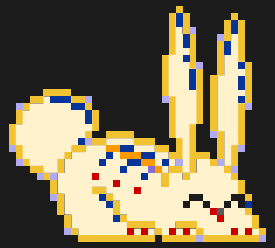
➔ Move Ordering

- What: Speeds up the AI by guessing which moves are best before checking them fully.
- How: We calculate a quick heuristic score for all moves and sort them. Checking good moves first makes Alpha-Beta pruning much more effective.

➔ Selective Deepening (Hard Mode)

- What: Allows "Hard" difficulty to be smart but still responsive (fast).
- How:
  - The Top 6 moves are searched to Full Depth (Deep).
  - The remaining moves are searched to Shallow Depth (Fast).
  - This avoids wasting time on obviously bad moves.

```cpp
inline Move getBestMove(const GameState& state, Difficulty difficulty, const std::atomic_bool* cancel = nullptr) {
    ... rest of implementation

    // Move Ordering
    for (const auto& move : moves) {
        GameState next = rootState.clone();
        applyMove(next, move);
        // Use the difficulty-specific evaluation
        int heuristic = evaluate(next, perspective, difficulty);
        scored.push_back({ move, heuristic });
    }

    std::sort(scored.begin(), scored.end(), [](const ScoredMove& a, const ScoredMove& b) {
        return a.heuristic > b.heuristic;
    });

    Move bestMove = scored.front().move;
    int bestScore = std::numeric_limits<int>::min();

    int primaryDepth = std::max(0, searchDepth - 1);
    int shallowDepth = std::max(0, primaryDepth - 1);

    // Variable Depth Logic for Hard: Only search the top 6 moves deeply
    std::size_t deepSlots = (difficulty == Difficulty::Hard && scored.size() > 6) ? 6 : scored.size();

    for (std::size_t idx = 0; idx < scored.size(); ++idx) {
        const auto& entry = scored[idx];
        GameState next = rootState.clone();
        applyMove(next, entry.move);

        int depthForMove = primaryDepth;
        if (difficulty == Difficulty::Hard && idx >= deepSlots) {
            depthForMove = shallowDepth;
        }

        int score = minimax(next, depthForMove, std::numeric_limits<int>::min(),
            std::numeric_limits<int>::max(), maximizingPlayer, perspective, moveCap, difficulty, cancel);

        if (score > bestScore) {
            bestScore = score;
            bestMove = entry.move;
        }
    }

    return bestMove;
}
```



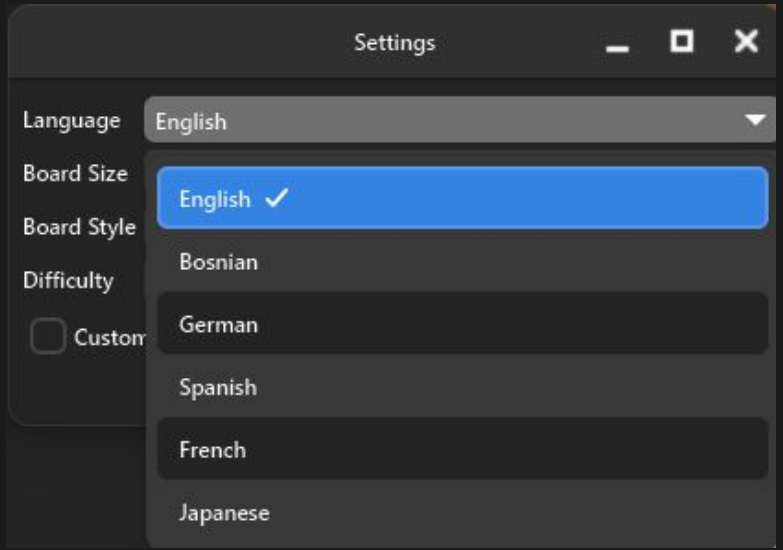
04

FEATURES

Available Languages

- The game is designed to be inclusive and it offers full support for diverse global communities.
- Multilingual support expands the potential user base significantly.
- Our application currently features professional localization for six major languages:

➔ English
➔ Bosnian
➔ German
➔ Spanish
➔ French
➔ Japanese (Kanji)



Original Features vs Additional Features

- ✓ **Core Game Rules:** Queen and Arrow placement, Path logic.
- ✓ **AI Engine:** Minimax with Alpha-Beta pruning.
- ✓ **Heuristics:** Spatial Influence & Territory Score
- ✓ **Difficulty Levels:** Easy, Medium, and Hard.
- ✓ **Board Sizes:** Small (6x6), Medium (8x8), Large (10x10)
- ✓ **Basic GUI:** Responsive interface, interactive Queens, move highlights.
- ✓ **Primary Themes:** Wooden and Black & White board styles.
- ✓ **Audio/Visuals:** Core animations for movement/explosions and basic sound effects.
- ✓ **Dual Language Support:** Interface available in Bosnian and English.

- ✓ **Expanded Theme Library:** Six additional visual styles.
- ✓ **Custom Theme Creator:** Integrated color pickers allowing users to define their own tile colors.
- ✓ **Additional Modes:** PvP and Spectator mode
- ✓ **Multi-Language Expansion:** Added support for total 6 languages.
- ✓ **Undo System:** A state-stack implementation that allows players to take back their last move.
- ✓ **Stop AI Functionality:** Safety feature to stop deep calculations safely.
- ✓ **Animation Speed Control:** Control of the pace of visual effects.
- ✓ **Move History Log:** A text-based record of all actions during the match.
- ✓ **Page System:** A three-page interface layout for Game, History, and Rules.
- ✓ **Full Compatibility:** App is compatible with windows, mac-os and linux

THANK YOU FOR YOUR ATTENTION